

A Holistic Approach to Wheeled Robot Design: Mechanical Structure and Control Strategies

Alikha Balan
Robotics and Automation
Toc H Institute of Science and
Technology, APJAKTU
Ernakulam, Kerala, India
alikhahalan04@gmail.com

Grigari T. Jose
Robotics and Automation
Toc H Institute of Science and
Technology, APJAKTU
Ernakulam, Kerala, India
grigari03@gmail.com

Jeevan Raju
Robotics and Automation
Toc H Institute of Science and
Technology, APJAKTU
Ernakulam, Kerala, India
naveejujar@gmail.com

Siddarth R
Robotics and Automation
Toc H Institute of Science and
Technology, APJAKTU
Ernakulam, Kerala, India
siddarthr2468@gmail.com

Georgina Binoy Joseph
Robotics and Automation
Toc H Institute of Science and
Technology, APJAKTU
Ernakulam, Kerala, India
georgina@tistcochin.edu.in

Abstract— A holistic approach to the design of various aspects of an agricultural differential drive robot for use in greenhouses is presented. The mechanical structure used along with design considerations for the motors used are elaborated. Three different controllers for path-following tasks in differential drive robots: Pure Pursuit along with Proportional Integral Derivative (PID) control, Stanley along with PID control, and Model Predictive Control (MPC) are used to determine control commands, including linear and angular velocities. The suitability of the controllers in terms of errors in path following are studied.

Keywords—*Robot Design and Navigation, Pure Pursuit, PID control, Stanley Controller, MPC Controller, Path Planning, Differential Drive Robot*

I. INTRODUCTION

Differential Drive Robots offer a simple and robust technique to implement wheeled mobile robots moving on engineered surfaces. Motion controllers ensure that the differential drive robot follow the given path minimizing errors. As the robots are driven by independently driven wheels, it can be challenging to control. The motion controller module ensures that the robot's actual motion follows its desired trajectory by controlling the wheel velocities to obtain a stable and responsive system.

Various controllers have been employed, each with unique advantages. The PID controller ensures stable formation maintenance, while Stanley controller excels in precision path tracking and the Pure Pursuit controller offers flexibility and responsiveness during manoeuvring [1]. The Pure Pursuit controller offers flexibility and responsiveness during navigation by adjusting steering angles for smooth path following. The PID controller, however, focuses on reducing positional errors to ensure precise and stable navigation [2]. The Stanley controller is strong and provides stable performance in various conditions.

Although Model Predictive Control (MPC) requires more computation and accurate models, it delivers optimal control by considering future states and constraints [3].

The robot's mechanical structure features a differential drive system with enhanced control for navigating complex environments [4]. PID and Pure Pursuit controllers are studied widely for effectiveness and simplicity and their performance in complex environments for autonomous vehicles [5]. Linear time varying MPC and Pure Pursuit controllers' evaluation and their performance, limitations and advantages for autonomous vehicle navigation are presented in [6]. Innovations in Smart Path Generation using MPC contributes to improved adaptability in dynamic scenarios [7], while experiments with Pure Pursuit for four wheeled robots also enhance manoeuvrability and path following efficiency [8].

The Stanley controller method was the path following approach used by Stanley, the autonomous vehicle of the 2005 DARPA challenge from Stanford university [9]. In [10], multiple paths tracking methods are compared to provide valuable insights on how different controllers perform. Optimisation controllers like LQR and MPC provide an automated design and optimization procedure.[11]. MPC can also be used to optimise look ahead distances in Pure pursuit controllers to further smoothen the path tracking process.[12]

In this paper, the performances of Pure Pursuit, Stanley and MPC controllers are investigated to determine the most optimum controller for the proposed system.

II. METHODOLOGY

The mechanical design of the differential drive robot was developed to ensure precise movement control. It uses high-quality components and a strong frame for durability. The robot has a differential drive system, with a castor wheel, and the wheels are powered by high-torque, high-

precision DC geared motors. This design ensures smooth, efficient movement and easy maneuvering.

A. Mechanical Structure

The robot's frame is made up of 4040 aluminum extrusion which is paired with T-slots which are easy to assemble and disassemble. It provides good structural strength and provide stability to the robot. The dimensions of the robot include 400 mm overall height including the camera mount, 350 mm width, and 150 mm height for the base frame (Fig. 1. and Fig. 2.). The robot uses two large bottom wheels and one castor wheel for stability. The components are attached together using 40x40 L-clamps and M6 bolts. A Polycarbonate sheets is fit onto the top and bottom of the robot using screws which provides it with protection. The assembly process is done using tools such as a drill for securing the polycarbonate sheets, L-brackets used for ensure structural integrity, and self-screws used for good attachment without drilling holes. The materials, design, and tools combined to give a durable and functional robotic structure.

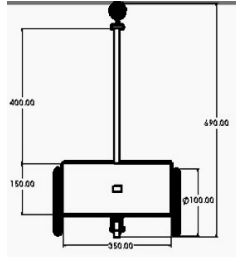


Fig. 1. Front View

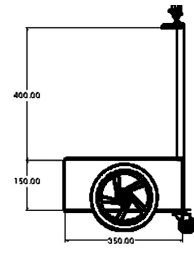


Fig. 2. Side View of Robot

B. Wheel and Motor Integration

The robot is equipped with a castor wheel and two wheels with two independently driven wheels forming the differential drive system. The driven wheels are paired with high-torque, high-precision DC geared motors that delivers the necessary power and control for both straight-line movement and turning. The castor wheel, which supports the robot, allows for rotation around the vertical axis, complementing the differential drive system and enabling sharp turns and precise manoeuvring. The selection of the motors was done for a payload $W = 30$ kg. The parameters for motor torque calculation include mass m , velocity of robot v , wheel radius r , angular velocity $\omega = v/r$, acceleration a , Coefficient of Rolling friction $\mu = 0.02$, Max angle of inclination, $\theta = 5^\circ$. Suitable values are considered for these parameters according to the application.

$$\text{Force } f = (m \times a) + (\mu \times m \times g + mg \sin(\theta)) \quad (1)$$

$$\text{Torque } T = f \times r \quad (2)$$

Considering a factor of safety of 4

$$\text{Torque } F_{os} = T \times 4 \quad (3)$$

$$\text{Power} = \text{Torque } F_{os} \times \text{velocity} \quad (4)$$

Based on the application in this case an agricultural robot for use in green houses, the above calculations resulted in the selection of an RCMS DC Geared Motor with 27kgcm

Holding Torque, No-load current of 800 mA and Load current of up to 7.5 A(Max).

C. System Design

The proposed system comprises of an agricultural mobile robot which is operated in a green house. It consists of motor control, sensing, and actuation for automation tasks. The block diagram in Fig. 3. illustrates the structure and interconnections of the system. The robot uses a Raspberry Pi 4b powered by 5V power bank as the main controller of the system. A webcam is used for capturing visual data which is processed by the Raspberry Pi using computer vision techniques. The processed information is then used to control the robot's movements and other components like the nozzle and pump.

Motion control is achieved with an Arduino Mega. Two DC 12V 100 RPM Johnson Quad Encoder Motors are connected to Rhino DC Servo Drivers (RMCS-2303). These are controlled using PWM signals that are generated by the Arduino. A 12V, 7.2 Ah lithium-ion battery powers the system. For actuation, an Arduino Uno manages a single channel 5V relay module, which operates a pump linked to a nozzle, ensuring precise liquid dispensing for agricultural applications.

For vision-based tasks, a Raspberry Pi is paired with a webcam. To improve processing efficiency, an external computer equipped with an Nvidia RTX 3050 GPU and a Ryzen 7 5600H CPU handles tasks such as crop analysis and image-based decision-making.

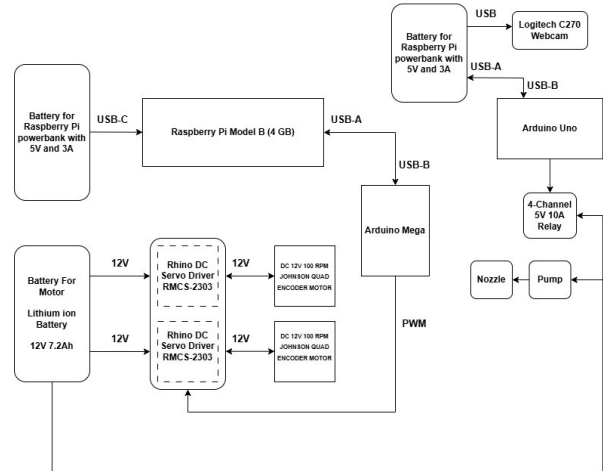


Fig. 3. Block Diagram of the Hardware System

III. CONTROLLERS

A. PID + Pure Pursuit controller

For accurate and smooth path tracking, the Pure Pursuit Algorithm calculates the required angular velocity to steer the robot towards the lookahead point in the path, even in varying environment the robot position and orientation can be dynamically adjusted to maintain a stable path tracking (Fig. 4.). The PID controller regulates the linear velocity to minimize the errors in velocity or distance to ensure consistent motion. These two controllers work together for precise navigation, helps the robot to handle complex path and adapt to disturbances and uncertainties in operation. This type of controller is easy to tune and is computationally lightweight, making it very suitable for low powered devices like Arduino, Raspberry, etc.

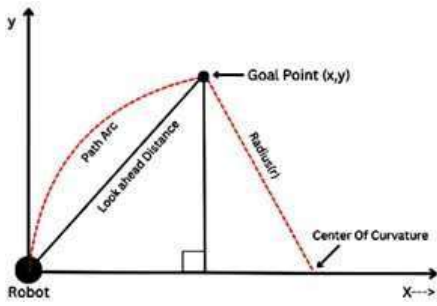


Fig. 4. Illustration of Pure Pursuit Algorithm

Let x_{goal} and y_{goal} be coordinates of the goal point and x_{robot} and y_{robot} be coordinates of the robot.

Lookahead point calculation:

$$\Delta x = x_{goal} - x_{robot}, \Delta y = y_{goal} - y_{robot} \quad (5)$$

$$\gamma = \text{atan2}(\Delta y, \Delta x) \quad (6)$$

Curvature:

$$\kappa = \frac{2\Delta y}{L^2} \quad (7)$$

Steering angle:

$$\delta = \arctan(L \cdot \kappa) \quad (8)$$

Angular velocity:

$$\omega = v \cdot \kappa \quad (9)$$

For PID control law:

$$v = K_p e_v + K_i \int e_v dt + K_d \frac{de_v}{dt} \quad (10)$$

where $e_v = v_{desired} - v_{current}$

$v_{desired}, v_{current}$ = desired and current velocity

v = control output of PID controller

L = Look ahead Distance

γ = Angle of robot's Heading

κ = Rate of change in robot's direction

K_p, K_i, K_d are Proportional, Integral and Derivative gain respectively

B. PID + Stanley Controller

Stanley controller is a path tracking algorithm that adjusts the steering angle to minimize the cross-track error (e_{CT}) and heading error (C_e) (shown in Fig. 5.). It was developed for Stanford University's Stanley Robot. The PID controller ensures that the linear velocity remains stable and consistent. It is easy to implement and is widely used in autonomous vehicles especially for structured environments like roads and parking lots. Its robust to small disturbances but it can be prone to oscillations at low speeds.

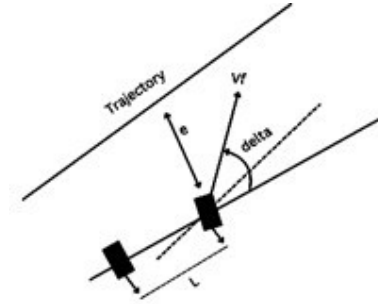


Fig. 5. Illustration of Stanley Controller

Cross-track-error e_{CT} ,

$$\sqrt{(x_{goal} - x)^2 + (y_{goal} - y)^2} \quad (11)$$

Heading-error,

$$C_e = C_{Path} - C_{robot} \quad (12)$$

Steering-Angle,

$$\delta = C_e + \tan^{-1} \left(\frac{K_s e_{CT}}{v} \right) \quad (13)$$

Angular-velocity,

$$\omega = \frac{v \cdot \sin(\delta)}{L} \quad (14)$$

where C_{Path} = Direction of path

C_{robot} = Direction of robot

C_e = Heading Error

K_s = Gain parameter for cross track error correction

L = Distance between front and rear axles

C. MPC Controller

MPC optimizes the control inputs (v, ω) over a finite time horizon by solving a constrained optimization

problem. It minimizes a cost function based on predicted trajectory errors while adhering to system dynamics and constraints. It offers the highest accuracy and flexibility at the cost of complexity and computational load.

Cost Function:

$$J = \sum_{k=1}^N (Q_x(x_k - x_{ref})^2 + Q_y(y_k - y_{ref})^2 + Q_{\dot{x}}(\dot{x}_k - \dot{x}_{ref})^2 + R_v v_k^2 + R_{\omega} \omega_k^2) \quad (15)$$

Kinematic Model:

$$\dot{x} = v \cos(\theta), \quad \dot{y} = v \sin(\theta), \quad \dot{\theta} = \omega \quad (16)$$

Optimal Control Inputs:

$$[v^*, \omega^*] = \text{argmin}_n(J) \quad (17)$$

where $x_k, y_k = x$ and y coordinates of the robot
 $\theta_k =$ robot's heading
 $v_k =$ linear velocity at step k
 $\omega_k =$ angular velocity at step k

$Q_x, Q_y, R_v, R_{\omega} =$ weighting factor that penalizes deviations in x, y , linear and angular velocity respectively

$x_{ref}, y_{ref}, C_r =$ Reference values for the robot's position

IV. RESULTS AND DISCUSSION

A. Robot Structure

The structure of the robot has been constructed and integrated with the motor system (Fig. 6.). This design incorporates all the necessary components including the mechanical structure, motors and controller with interfaces.



Fig. 6. Structure of the robot

B. Controller Performance in Path Following

Figs. 7, 8 and 9 show the differential drive robot performance in navigating a path from a specified start point to predefined goal point. The dotted line represents the desired path, while the actual path followed by the robot is given by the solid line with the x - and y -axes indicating horizontal and vertical distances. The figures show essential markers, including the start position (green circle), the goal position (red cross), the planned trajectory (solid cyan line), and the actual path taken by the robot (black dashed line).

In Fig. 7. the robot follows the desired path using PID and Pure Pursuit controller. As the path starts to get more

complex, the robot's trajectory starts to diverge. Many factors can contribute to this like wheel slippage, improper path planning or even controller limitations. It could be better to optimize and refine the controller so as to obtain an efficient tracking algorithm which can be implemented in complex environments.

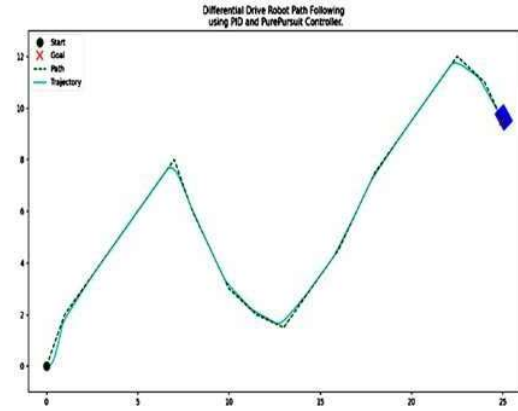


Fig. 7. Path following using PID and Pure Pursuit Controller

Fig. 8. shows the performance of the differential drive robot with PID controller and Stanley controller in navigating a path from a specified start point to predefined goal point. Even though the robot follows the path effectively initially, deviations occur where there are sharp turns and complex curves, caused by imperfect path planning, controller limitations, and environmental disturbances. Further optimization and refinements are needed for precise and robust path tracking in the real-world scenarios which are challenging environments.

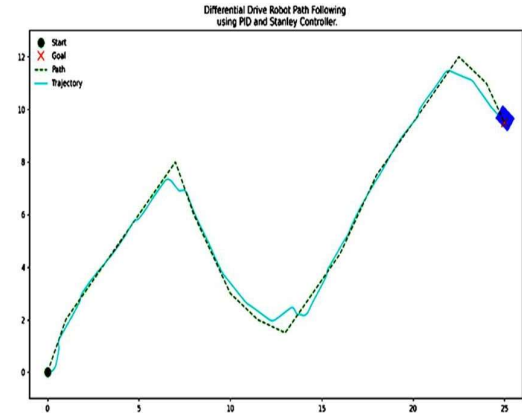


Fig. 8. Path following using PID and Stanley Controller

Figs. 9. depicts the path-following performance of a differential drive robot using a Model Predictive Control (MPC) controller. The MPC controller predicts future states, and allows the robot to reach the goal. Nevertheless, deviations may occur due to sensor noise, actuator limitations, and environmental disturbances. Further optimization is needed to improve performance in real-world environments.

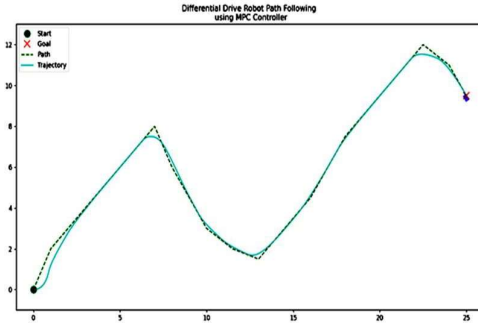


Fig. 9. Path following using MPC Controller

The error metrics used for comparing the performance of the controllers are:

Tracking Error,

$$E_{\text{track}} = \sqrt{(x_r - x_p)^2 + (y_r - y_p)^2} \quad (18)$$

Maximum Error,

$$E_{\text{max}} = \max_i \sqrt{(x_r - x_{pi})^2 + (y_r - y_{pi})^2} \quad (19)$$

Minimum Error,

$$E_{\text{min}} = \min_i \sqrt{(x_r - x_{pi})^2 + (y_r - y_{pi})^2} \quad (20)$$

Average Error,

$$E_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N \sqrt{(x_r - x_{pi})^2 + (y_r - y_{pi})^2} \quad (21)$$

Standard Deviation,

$$\sigma_{E_{\text{track}}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (E_{\text{track}_i} - E_{\text{avg}})^2} \quad (22)$$

where (x_r, y_r) : Robot's current position
 (x_p, y_p) : Path point coordinates.
 N : Total number of path points.

The results for Stanley + PID, Pure Pursuit + PID, and MPC controllers given in Tables 1, 2 and 3 for three different paths show effective path-following performance with slight differences in the tracking errors:

- **Stanley + PID:** Performs alright with consistent results. It is seen to struggle with sharp corners while undergoing speed variations. The average error varies rapidly to 0.48m in the third trial. Fine tuning and refinements to PID can increase the efficiency of the robot path tracking.

- **Pure Pursuit + PID:** Shows slightly better performance than Stanley + PID, with an average error of 0.46 meters in later trials. It works well on smoother paths, but require optimization for better accuracy.
- **MPC:** Shows stability with low minimum errors (0.00 meters), but its average error (0.51 meters) is slightly higher than Pure Pursuit + PID. MPC is better for dynamic environment and non-linear path but can be fine-tuned for straight line path following.

Overall, Pure Pursuit + PID and MPC performs better than Stanley + PID controller.

TABLE I: ERRORS - PATH 1

Controller	E _{max}	E _{min}	E _{avg}
Pure Pursuit +PID	1.11	0.01	0.52
Stanley +PID	1.13	0.06	0.59
MPC	1.18	0.00	0.54

TABLE II: ERRORS - PATH 2

Controller	E _{max}	E _{min}	E _{avg}
Pure Pursuit + PID	1.06	0.01	0.51
Stanley+ PID	1.07	0.00	0.54
MPC	1.09	0.03	0.51

TABLE III: ERRORS - PATH 3

Controller	E _{max}	E _{min}	E _{avg}
Pure Pursuit + PID	1.05	0.01	0.46
Stanley + PID	1.04	0.00	0.48
MPC	1.05	0.03	0.46

V. CONCLUSION

The robot is built with a two wheeled differential drive configuration featuring an aluminum frame and is provided with a castor wheel for balance. It uses a Raspberry Pi 4b for motion controlling along with an Arduino Mega which controls the motors via the motor drivers. The robot is built to be used in a greenhouse setting which has a relatively straight or curved paths between the crop rows. The robot

is operated at lower speeds to ensure safety and to avoid damage to plants.

Three control strategies – Pure Pursuit controller with PID controller, Stanley controller with PID controller and a Model Predictive Controller (MPC) were studied. The controllers were compared based on the lateral tracking error they produced. They were tested on 3 different paths and the tracking error was calculated. It was found that both MPC and Pure Pursuit along with PID were far better in path following than Stanley with PID. As a result, the Pure Pursuit along with PID controller was chosen for path following. The MPC controller was not chosen due to the computation limits of the Raspberry Pi 4b and also due to the added complexity it presents to the system. Pure Pursuit along with PID is sufficient for low-speed greenhouse operations where high precision is not critical. Moreover, it is a practical choice for real-world applications because it is fast, efficient, and easy to implement on various robotic platforms, even those with limited processing power. It handles sudden changes well, making quick adjustments to stay on track. Compared to Stanley + PID and MPC, which need more computing power, this controller offers a good balance of accuracy and efficiency, making it ideal for resource-limited environments where responsiveness and energy use matter.

ACKNOWLEDGMENT

The project is funded by the Center of Engineering and Research Development (CERD), APJ Abdul Kalam Technological University, under the Financial Assistance to Student Projects 2024-25 Funding scheme.

REFERENCES

- [1] W. Zhu, Y. Zhang, W. Kong, F. Jiang, and P. Ji, "A versatile control method for multi-agricultural machine cooperative steering applicable to two steering modes," in *Proc. World Electr. Veh. J.*, 2024, p. 126.
- [2] A. Jiang and T. Ahamed, "Navigation of an autonomous spraying robot for orchard operations using LiDAR for tree trunk detection," in *Proc. Sensors*, 2023, p. 4808.
- [3] J. Lee and S. Yim, "Comparative study of path tracking controllers on low friction roads for autonomous vehicles," in *Proc. Machines*, 2023, p. 403.
- [4] M. Liu, M. Chen, Z. Wu, B. Zhong, and W. Deng, "Implementation of intelligent indoor service robot based on ROS and deep learning," in *Proc. Machines*, 2024, p. 256.
- [5] H. Jain and P. Babel, "A comprehensive survey of PID and pure pursuit control algorithms for autonomous vehicle navigation," in *Proc. arXiv*, 2024.
- [6] S. Lindenfors and S. Rahmanian, "Comparison of linear time varying model predictive control and pure pursuit control for autonomous vehicles," in *Proc. Conf.*, 2024.
- [7] Y. Wu, S. Li, Q. Zhang, K. Sun-Woo, and L. Yan, "Route planning and tracking control of an intelligent automatic unmanned transportation system based on dynamic nonlinear model predictive control," in *IEEE Trans. Intell. Transp. Syst.*, 2022, pp. 16576–16589.

- [8] G. Bayar, "Trajectory tracking control of unmanned ground vehicles in mixed terrain," *Ph.D. dissertation*, Middle East Technical Univ., Turkey, 2012.
- [9] S. Thrun et al., "Stanley: The robot that won the DARPA Grand Challenge," in *Proc. J. Field Robot.*, 2006, pp. 661–692.
- [10] J. M. Snider, "Automatic steering methods for autonomous automobile path tracking," *Robotics Inst.*, Carnegie Mellon Univ., 2009.
- [11] J. Liu, Z. Yang, Z. Huang, W. Li, S. Dang, and H. Li, "Simulation performance evaluation of pure pursuit, Stanley, LQR, MPC controller for autonomous vehicles," in *Proc. IEEE Int. Conf. Real-Time Comput. Robot. (RCAR)*, Jul. 2021, pp. 1444–1449.
- [12] S. Kim, J. Lee, K. Han, and S. B. Choi, "Vehicle path tracking control using pure pursuit with MPC-based look-ahead distance optimization," in *Proc. IEEE Trans. Veh. Technol.*, 2023, pp. 53–66.